

Design Patterns

Design Pattern là gì?

Design patterns are typical solutions to commonly occurring problems in software design. They are like pre-made blueprints that you can customize to solve a recurring design problem in your code.

- ☕ Không phải đoạn code copy–paste được
- ☕ Là **khuôn mẫu tư duy** tổng quát cho một vấn đề thiết kế
- ☕ Được **tùy biến** để phù hợp với từng chương trình cụ thể

Pattern $\neq$ Algorithm

Algorithm

- ☕ Tập hợp **bước cụ thể** để đạt mục tiêu
- ☕ Giống như **công thức nấu ăn** — có trình tự rõ ràng
- ☕ Kết quả **xác định** và **lặp lại được**

Design Pattern

- ☕ Mô tả **giải pháp cấp cao** cho một vấn đề
- ☕ Giống như **bản thiết kế** — bạn tự quyết cách triển khai
- ☕ Cùng một pattern, **code có thể khác nhau** ở mỗi dự án

Cấu trúc một Design Pattern

4 thành phần:

- ☕ **Intent** — Mục đích
- ☕ **Motivation** — Động lực / vấn đề
- ☕ **Structure** — Cấu trúc lớp / đối tượng
- ☕ **Code Example** — Ví dụ minh họa

Thông tin bổ sung

- ☕ **Applicability** — Khi nào nên dùng
- ☕ **Implementation steps** — Các bước triển khai
- ☕ **Relations** — Liên hệ với các pattern khác

Một pattern tốt cung cấp đủ cả 4 phần cốt lõi lẫn thông tin bổ sung để bạn hiểu và áp dụng đúng.

Lịch sử hình thành

Christopher Alexander (1977)

- ☕ Kiến trúc sư người Áo–Mỹ
- ☕ Xuất bản cuốn **"A Pattern Language: Towns, Buildings, Construction"**
- ☕ Đề xuất khái niệm **"ngôn ngữ mẫu"** (pattern language) trong thiết kế đô thị
- ☕ Mỗi *pattern* mô tả một vấn đề **lặp lại** trong môi trường sống và cách giải quyết nó

Ý tưởng chính: Khi một giải pháp được lặp lại nhiều lần trong nhiều dự án → ai đó đặt tên cho nó và ghi lại → **Pattern ra đời**

Gang of Four – GoF (1994)

Cuốn sách kinh điển

"Design Patterns: Elements of Reusable Object-Oriented Software"

- ☕ Xuất bản năm **1994**
- ☕ Áp dụng khái niệm pattern vào **lập trình hướng đối tượng**
- ☕ Giới thiệu **23 patterns** kinh điển
- ☕ Trở thành **bestseller** ngay lập tức

Bộ tứ tác giả

Tác giả	Vai trò
Erich Gamma	Kiến trúc sư chính
Richard Helm	Structural patterns
Ralph Johnson	Behavioral patterns
John Vlissides	Creational patterns

Phong trào lan rộng

- ☕ Hàng chục OOP patterns **mới** tiếp tục được khám phá
- ☕ Tư duy **pattern approach** lan sang **nhiều lĩnh vực** khác ngoài OOP
 - 🌿 Kiến trúc phần mềm (Microservices, Event-Driven...)
 - 🌿 Giao diện người dùng (UI Patterns)
 - 🌿 Hệ thống phân tán (Distributed Systems Patterns)
- ☕ Ngày nay: pattern không còn giới hạn ở OOP

"Pattern là giải pháp đã được kiểm chứng qua thực tế — không ai phát minh ra chúng, người ta chỉ khám phá và đặt tên cho chúng."

Why

The truth is that you might manage to work as a programmer for many years without knowing about a single pattern. A lot of people do just that. Even in that case, though, you might be **implementing some patterns without even knowing it.**

- ☕ Nhiều lập trình viên kỳ cựu dùng pattern **mà không biết tên**
- ☕ Biết tên pattern giúp bạn **trao đổi nhanh hơn** với đồng nghiệp
- ☕ Học pattern giúp bạn **tư duy có hệ thống** hơn về thiết kế

#1: Bộ công cụ đã được kiểm chứng

Toolkit of Tried Solutions

- Design patterns là **tập hợp giải pháp đã được thử nghiệm** cho các vấn đề phổ biến trong thiết kế phần mềm
- Dù bạn chưa gặp những vấn đề đó, **hiểu pattern dạy bạn tư duy OOP**
- Tránh phải *"phát minh lại bánh xe"* mỗi lần gặp bài toán quen thuộc

Lợi ích thực tế

- Giải pháp **đã được nhiều dự án kiểm chứng**
- Tập trung vào **vấn đề thực sự** thay vì mày mò cách giải
- Code dễ **bảo trì** và **mở rộng** hơn
- Giảm thiểu **kỹ thuật nợ** (technical debt)

#2: Ngôn ngữ chung của lập trình viên

Common Language

Patterns định nghĩa **ngôn ngữ chung** để cả team giao tiếp hiệu quả hơn.

Ví dụ thực tế

Thay vì giải thích dài dòng:

"Hãy tạo một lớp chỉ có một thể hiện duy nhất, và cung cấp một điểm truy cập toàn cục đến nó..."

Chỉ cần nói:

"Oh, just use a Singleton for that"

→ Cả team **hiểu ngay** — không cần giải thích thêm!

Tóm lại: Tại sao nên học Pattern?

Lý do	Giải thích
Toolkit đã kiểm chứng	Giải pháp được thực tế xác nhận
Nâng cao tư duy OOP	Học cách giải quyết vấn đề theo nguyên tắc
Ngôn ngữ chung	Giao tiếp nhanh, rõ ràng trong team
Tái sử dụng	Áp dụng lại ở nhiều project
Code chất lượng cao	Dễ bảo trì, mở rộng, đọc hiểu

Các ý kiến trái chiều

"It seems like only lazy people haven't criticized design patterns yet."

— refactoring.guru

Có **3 luồng ý kiến trái chiều chính** đối với design patterns:

-  **Kludges for a weak programming language**
-  **Inefficient solutions**
-  **Unjustified use**

#1: Sự chấp vá cho ngôn ngữ lập trình yếu

Kludge for Weak Languages

"Nhu cầu dùng patterns thường xuất hiện khi người ta chọn ngôn ngữ hoặc công nghệ thiếu mức độ trừu tượng cần thiết."

— Paul Graham, *Revenge of the Nerds*

Pattern trở thành **"siêu năng lực"** hỗ trợ cho ngôn ngữ lập trình.

Ví dụ

Pattern **Strategy** trong Java:

```
interface SortStrategy {  
    void sort(int[] arr);  
}
```

Trong Python / JavaScript hiện đại:

```
# Chỉ cần truyền function!  
sort(data, key=lambda x: x.price)
```

→ Ngôn ngữ mạnh hơn → ít cần pattern hơn

#2: Giải pháp kém hiệu quả

Inefficient Solutions

- ☕ Patterns **hệ thống hóa** các cách tiếp cận phổ biến
- ☕ Nhiều người xem đây là **giáo điều** (dogma)
- ☕ Họ implement "**đúng theo sách**" mà **không điều chỉnh** theo ngữ cảnh dự án

Hậu quả

- ☕ Pattern trở nên **phức tạp hóa** không cần thiết
- ☕ Không giải quyết đúng **vấn đề thực tế** của dự án
- ☕ Codebase trở nên **khó hiểu** hơn thay vì dễ hiểu hơn

Pattern là **hướng dẫn**, không phải **quy tắc cứng**.

#3: Over-engineering

"If all you have is a hammer, everything looks like a nail." → **Nguyên tắc:** Chỉ dùng pattern khi **thực sự cần thiết**.

Unjustified Use

- Người mới học pattern thường **áp dụng mọi nơi**, kể cả những nơi mà **code đơn giản hơn** hoàn toàn đủ
- Đây là **anti-pattern** phổ biến nhất trong cộng đồng lập trình

Dấu hiệu lạm dụng

- Thêm nhiều **abstract layer** không cần thiết
- Code có pattern nhưng **không giải quyết** vấn đề rõ ràng
- Team mới cần nhiều thời gian để **onboard**
- Refactor liên tục

Sai lầm thường gặp

- ☕ Học pattern xong → áp dụng tất cả
- ☕ Xem pattern là **giải pháp cho mọi thứ**
- ☕ Không xem xét ngữ cảnh và trade-offs
- ☕ Bỏ qua sự đơn giản (KISS principle)

Cách tiếp cận đúng

- ☕ Hiểu **vấn đề** trước khi nghĩ đến pattern
- ☕ Pattern là **tool**, không phải **mục tiêu**
- ☕ Ưu tiên **code dễ đọc** hơn code "đúng pattern"
- ☕ Refactor khi thực sự thấy cần thiết

Phân loại

3 Nhóm Pattern Chính (GoF)

Nhóm	Mục đích	Từ khóa
Creational	Tạo đối tượng	Factory, Singleton, Builder...
Structural	Tổ chức cấu trúc	Adapter, Decorator, Facade...
Behavioral	Giao tiếp & trách nhiệm	Observer, Strategy, Command...

Creational Patterns

Creational patterns provide object creation mechanisms that increase **flexibility** and **reuse** of existing code.

- ☕ Kiểm soát **cách tạo ra** đối tượng
- ☕ Che giấu logic tạo đối tượng khỏi client code
- ☕ Tăng tính **linh hoạt** khi thay đổi loại đối tượng

Các pattern tiêu biểu

- ☕ Factory Method
- ☕ Abstract Factory
- ☕ Builder
- ☕ Prototype
- ☕ Singleton

Structural Patterns

Structural patterns explain how to assemble objects and classes into **larger structures**, while keeping these structures **flexible and efficient**.

- ☕ Kết hợp các lớp và đối tượng thành cấu trúc lớn hơn
- ☕ Vẫn giữ cấu trúc **linh hoạt** và **hiệu quả**

Các pattern tiêu biểu

- ☕ Adapter
- ☕ Bridge
- ☕ Composite
- ☕ Decorator
- ☕ Facade
- ☕ Flyweight
- ☕ Proxy

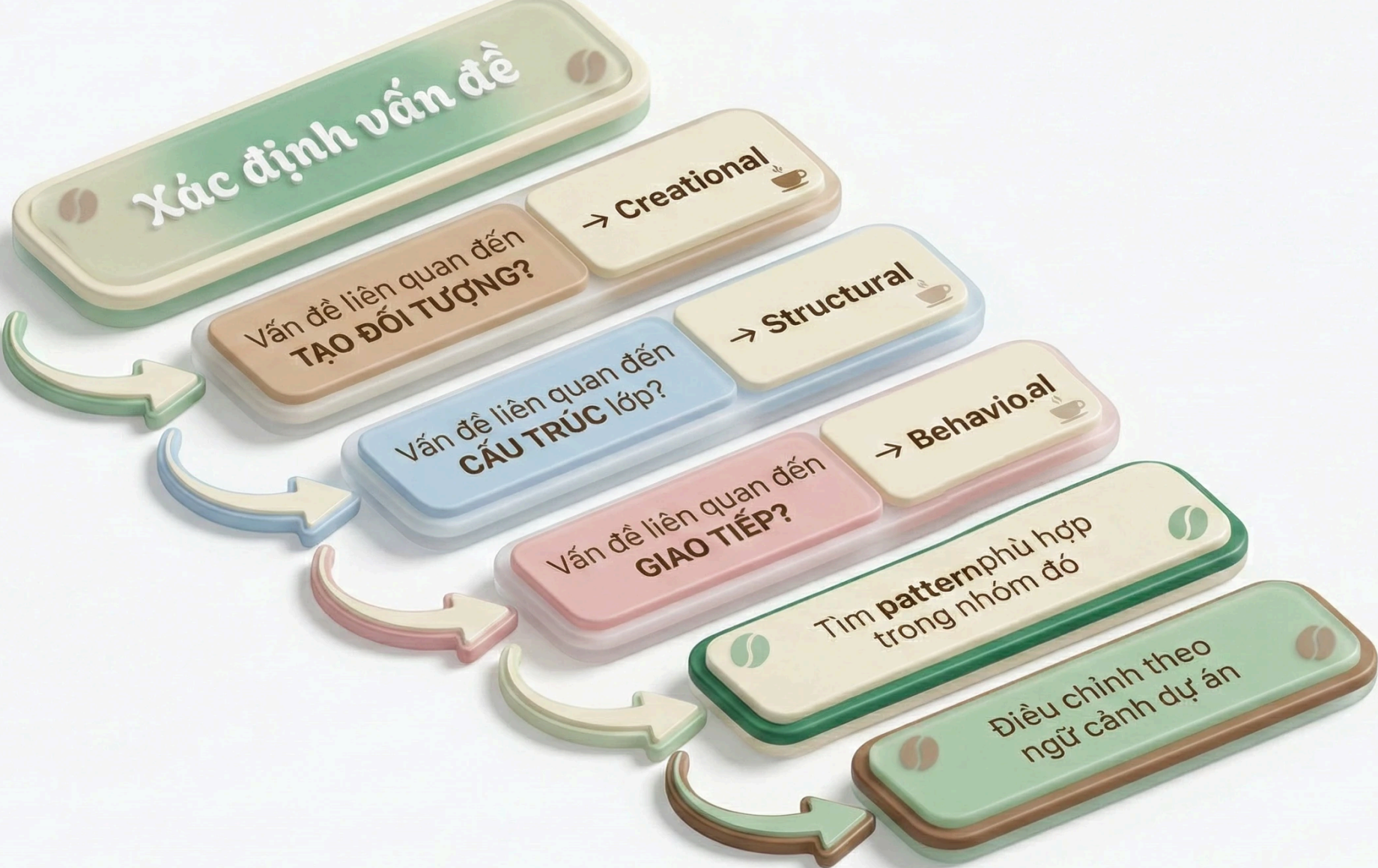
Behavioral Patterns

Behavioral patterns take care of **effective communication** and the **assignment of responsibilities** between objects.

- ☕ Định nghĩa cách các đối tượng **giao tiếp** với nhau
- ☕ Phân chia **trách nhiệm** rõ ràng giữa các lớp
- ☕ Quản lý **luồng xử lý** và **thuật toán**

Các pattern tiêu biểu

- ☕ Chain of Responsibility
- ☕ Command
- ☕ Iterator
- ☕ Mediator
- ☕ Observer
- ☕ State
- ☕ Strategy
- ☕ Template Method



Mối liên hệ với các nguyên lý SOLID

Nguyên lý	Pattern liên quan
Single Responsibility	Command, Strategy
Open/Closed	Decorator, Strategy, Observer
Liskov Substitution	Template Method
Interface Segregation	Facade, Adapter
Dependency Inversion	Factory, Abstract Factory, Builder

Ví dụ - Hệ thống thông báo

Xây dựng hệ thống gửi thông báo cho ứng dụng thương mại điện tử:

- ☕ Khi đặt hàng thành công → gửi **Email** + **SMS** + **Push Notification**
- ☕ Mỗi kênh gửi theo cách khác nhau
- ☕ Sau này có thể thêm kênh mới (Zalo, Telegram...)
- ☕ Người dùng có thể **chọn kênh** muốn nhận

Phân tích

Nhận diện vấn đề

- Nhiều kênh thông báo → cần **mở rộng dễ dàng**
- Người dùng chọn kênh → cần **linh hoạt khi runtime**
- Hành vi khác nhau nhưng cùng mục đích → cần **giao diện thống nhất**

Gợi ý Pattern

Vấn đề	Pattern phù hợp
Một sự kiện → nhiều người nhận	Observer
Chọn hành vi lúc runtime	Strategy
Gói lệnh gửi thành object	Command
Ẩn phức tạp hệ thống con	Facade

Áp dụng: Strategy Pattern

```
interface NotificationStrategy {  
    void send(String recipient, String message);  
}  
  
class EmailStrategy implements NotificationStrategy {  
    @Override  
    public void send(String recipient, String message) {  
        System.out.println("[EMAIL] → " + recipient + ": " + message);  
    }  
}  
  
class SMSStrategy implements NotificationStrategy {  
    @Override  
    public void send(String recipient, String message) {  
        System.out.println("[SMS] → " + recipient + ": " + message);  
    }  
}
```

```
class NotificationService {
    private NotificationStrategy strategy;

    public NotificationService(NotificationStrategy strategy) {
        this.strategy = strategy;
    }

    public void setStrategy(NotificationStrategy strategy) {
        this.strategy = strategy; // Đổi kênh lúc runtime!
    }

    public void notify(String recipient, String message) {
        this.strategy.send(recipient, message);
    }
}

// --- Sử dụng ---
NotificationService service = new NotificationService(new EmailStrategy());
service.notify("user@example.com", "Đơn hàng #1234 đã được xác nhận!");

service.setStrategy(new SMSStrategy());
service.notify("0901234567", "Đơn hàng #1234 đã được xác nhận!");
```


Áp dụng: Observer Pattern

```
import java.util.*;

class OrderEvent {
    private List<NotificationStrategy> observers = new ArrayList<>();

    public void subscribe(NotificationStrategy observer) {
        observers.add(observer);
    }

    public void notifyAll(String recipient, String message) {
        for (NotificationStrategy obs : observers) {
            obs.send(recipient, message);
        }
    }
}
```

```
// --- Sử dụng ---  
OrderEvent event = new OrderEvent();  
event.subscribe(new EmailStrategy());  
event.subscribe(new SMSStrategy());  
event.subscribe(new PushNotificationStrategy()); // Để thêm kênh mới!  
  
event.notifyAll("user@example.com", "Cảm ơn bạn đã đặt hàng!");
```

Trước (không dùng Pattern)

```
void sendNotification(User user, String msg) {  
    // Vô số if/else  
    if (user.wantsEmail()) {  
        sendEmail(user.getEmail(), msg);  
    }  
    if (user.wantsSms()) {  
        sendSms(user.getPhone(), msg);  
    }  
    if (user.wantsPush()) {  
        sendPush(user.getToken(), msg);  
    }  
    // Thêm kênh mới = sửa hàm này!  
}
```

Vấn đề: Vi phạm OCP, khó test, khó mở rộng

Sau (dùng Strategy + Observer)

```
// Thêm kênh mới chỉ cần:  
class ZaloStrategy implements NotificationStrategy {  
    @Override  
    public void send(String recipient, String message) {  
        System.out.println("[ZALO] → " + recipient + ": " + message);  
    }  
}  
  
// Đăng ký là xong!  
event.subscribe(new ZaloStrategy());
```

Lợi ích: Tuân thủ OCP, dễ test từng kênh độc lập

Checklist đánh giá

Khi áp dụng một Design Pattern, hãy tự hỏi:

- ☕ Pattern này **giải quyết đúng vấn đề** chưa?
- ☕ Code sau khi áp dụng có **dễ đọc hơn** không?
- ☕ Có dễ **thêm tính năng mới** mà không sửa code cũ không?
- ☕ Có dễ **viết unit test** cho từng thành phần không?
- ☕ Có đang **over-engineer** không? (Pattern có quá phức tạp cho bài toán không?)

Tài liệu tham khảo



<https://refactoring.guru/design-patterns>